

Окружение считается подготовленным, если интерпретатор запускается из `.venv`, библиотеки импортируются без ошибок, а `notebook` выполняет ячейки последовательно. Это делает практику воспроизводимой и отделяет зависимости курса от системной установки Python.

3. Минимальное введение в Python, NumPy и Pandas для анализа

Python хранит значения в объектах с динамическими типами, однако для анализа данных критично не столько объявление переменных, сколько структура массива и тип элементов. NumPy предоставляет однородные многомерные массивы `ndarray` и векторные операции, а Pandas — `Series` и `DataFrame` с индексами, именованными столбцами и смешанными типами данных.

Главный практический принцип — по возможности заменять построчные циклы векторными выражениями. Операция над целым столбцом читается как аналитическое правило и обычно эффективнее Python-цикла.

Практический блок 3.1. Векторное преобразование

В следующем фрагменте будет смоделирован типичный для анализа данных переход от исходной единицы измерения к производной. Три значения скорости ветра в м/с объединяются в массив, затем весь массив одним выражением переводится в км/ч по формуле $v_{\text{km/h}} = 3.6 \cdot v_{\text{m/s}}$. Функция `np.round(array, 2)` округляет все элементы результата до двух знаков после десятичной точки.

```
import numpy as np

wind_ms = np.array([1.14, 0.80, 0.97])
wind_kmh = wind_ms * 3.6
print(np.round(wind_kmh, 2))
```

Результат выполнения

```
[4.10 2.88 3.49]
```

Умножение выполняется сразу для всего массива. Точно тот же принцип используется ниже для столбца `DataFrame`: `df["wind_speed_ms"] * 3.6`.

Практический блок 3.2. Ключевые объекты Pandas

Сначала создадим минимальные примеры `Series` и `DataFrame`, после чего программа явно покажет их типы. Далее сгруппируем данные по типам и подсчитаем статистику.

`pd.Series()` создаёт одномерную индексированную структуру Pandas, а параметр `name` задаёт имя серии.

`pd.DataFrame()` формирует таблицу, в данном примере ей передаётся словарь, где ключи становятся именами столбцов, а списки — их значениями.

Цепочка `frame.groupby("type")["value"].sum()` сначала группирует строки по значениям столбца `type`, затем выбирает столбец `value` и вычисляет сумму внутри каждой группы.

```
s = pd.Series([10, 20, 30], name="value")
frame = pd.DataFrame({
    "type": ["A", "B", "A"],
    "value": [10, 20, 30]
})

print(type(s).__name__)
print(type(frame).__name__)
print(frame.groupby("type")["value"].sum())
```

Результат выполнения

```
Series
DataFrame
type
A      40
B      20
Name: value, dtype: int64
```

`Series` — один именованный столбец с индексом; `DataFrame` — набор согласованных `Series`. `groupby()` реализует типичную аналитическую операцию «разбить на группы → агрегировать».

4. Форматы файлового хранения данных и загрузка CSV

CSV — текстовый табличный формат без строгой схемы типов: значения хранятся строками, а типы восстанавливаются при чтении. JSON удобен для вложенных и полуструктурированных объектов, но при записи табличных данных часто занимает больше места. Parquet — бинарный колоночный формат, сохраняющий типы и ориентированный на аналитическое чтение отдельных столбцов. В последующих работах Spark будет особенно эффективно использовать Parquet.